Technical Help Guide: `KeywordValueService`

1. Purpose

`KeywordValueService` is the **centralized, high-performance, natural-key lookup service** for all configurable enums, master data, dropdown values, statuses, types, and lookup data in the application.

It provides:

- **ID-less architecture** using natural string codes
- Full consistency with `BaseModel` (audit fields, soft deletes, `is_active`)
- Automatic **UPPERCASE** normalization for all codes
- Hierarchical/tree support via `HasTreeStructure`
- Smart caching with automatic invalidation
- 100% backward compatibility with existing code

---

2. Architecture & Design Principles

- **KeywordMaster** table: Stores metadata about each keyword group (e.g. `VTRANS` for Vehicle Transmission)
    - Natural `code` (e.g. `VTRANS`, `FUEL_TYPE`)
    - Central control: `is_active`, description, extra_data, image, etc.
- **Keyvalue** table: Stores individual values
    - Composite natural key: `keyword_code` + `code` (e.g. `VTRANS-AUTO`, `VTRANS-MAN`)
    - All codes are **forced to UPPERCASE**

This design gives you:

- Central metadata management per keyword group
- Fast, clean lookups without integer IDs
- Full tree/hierarchy support
- Future extensibility

---

3. Backward Compatibility

All **existing public method signatures** remain unchanged and work exactly as before.

Old calls still work perfectly:

```
KeywordValueService::getValueId('fuel_type', 'DIESEL');    // returns inte
KeywordValueService::getValue('fuel_type', 'DIESEL');      // returns full
KeywordValueService::getEnum('fuel_type');                 // returns arra
```

---

4. Complete Function Reference

All methods are **static**.

Recommended New Methods (ID-less – Use these going forward)

Method	Signature	Return Type	Example	Return Example
`getCode()`	`getCode(string $keywordCode, string $valueCode, bool $activeOnly = true)`	`?string`	`KeywordValueService::getCode('VTRANS', 'AUTO')`	`"AUTO"`
`getByCode()`	`getByCode(string $keywordCode, string $valueCode, bool $activeOnly = true)`	`?Keyvalue`	`KeywordValueService::getByCode('VTRANS', 'AUTO')`	Full `Keyvalue` model
`getEnum()`	`getEnum(string $keywordCode, bool $activeOnly = true)`	`array`	`KeywordValueService::getEnum('VTRANS')`	`['AUTO' => 'Automatic', 'MAN' => 'Manual']`

Legacy / Backward Compatible Methods

Method	Signature	Return Type	Example	Return Example
`getValueId()`	`getValueId(string $keyword, string $key, bool $activeOnly = true)`	`?int`	`getValueId('fuel type', 'DIESEL')`	`45` (integer id)
`getValue()`	`getValue(string $keyword, string $key, bool $activeOnly = true)`	`?Keyvalue`	`getValue('fuel_type', 'DIESEL')`	Full model

Utility Methods

- `clearCache(?string $keywordCode = null)` — Clear cache for one keyword or all
- `getKeywordId(string $keyword)` — Legacy (returns master id)

---

5. Practical Use Cases & Examples

1. Dropdown / Select Box

```
<select name="transmission_code">
    @foreach(KeywordValueService::getEnum('VTRANS') as $code => $name)
        <option value="{{ $code }}">{{ $name }}</option>
    @endforeach
</select>
```

2. Saving Data (Recommended ID-less)

```
$transmissionCode = KeywordValueService::getCode('VTRANS', $request->trar

$vehicle->transmission_code = $transmissionCode;
$vehicle->save();
```

3. Validation

```
if (KeywordValueService::getCode('VTRANS', $request->transmission) === nu
    // Invalid transmission type
}
```

4. Checking Access / Permission

```
if (KeywordValueService::getCode('permit', $request->permit) === 'COMMERC
    // Allow commercial permit logic
}
```

5. Full Model Access

```
$value = KeywordValueService::getByCode('FUEL_TYPE', 'DIESEL');
echo $value->value;         // "Diesel"
echo $value->details;       // extra info
```

---

6. Best Practices

- **Always use** `getCode()` and `getByCode()` for new development
- Store only the `code` (e.g. `AUTO`) in your business tables

- Use `getEnum()` for all dropdowns/selects
- Call `KeywordValueService::clearCache()` after seeding or bulk updates
- All codes are automatically uppercased — no need to worry about case

---

We now have a clean, modern, ID-less, metadata-rich lookup system that is fully consistent with our User/Person/Employee/Vehicle architecture.